University Software Inventory Database Project Brief

Nick Tsai

1. Project Rationale

The university currently does not have one central system to track all the software it owns, licenses, rents, or manages. This makes it difficult for the IT department and Legal to identify what software is being used across the university, especially when preparing for audits. The goal of this project is to create a centralized software inventory database using a platform the university already licenses. The system will collect information from all fourteen academic and administrative units, including software names, versions, licensing agreements, departments, machines, and installation dates. The original goal is to complete the project within nine months and stay within the $85,000 budget. Before the annual audit, the database should contain records from all fourteen units, with missing or unverified information clearly identified. This follows the SMART approach described by Jewell (2026) by establishing a specific, measurable, realistic, and time-based goal.

2. Scope and Boundaries

In Scope: The project includes creating a centralized database to track university software, gathering information from the fourteen units, organizing existing licensing records, testing the system, and training the IT department to maintain it. The database will track software names, types, developers, versions, licensing agreements, departments, machines, and installation dates. The project will also include working with the college that maintains its own spreadsheet to incorporate its information into the central system.

Out of Scope: The project will not purchase a new platform, build a separate hardware inventory, renegotiate licensing agreements, or make purchasing decisions for individual departments. The college with the existing spreadsheet will not be forced to stop using it immediately. Instead, its records will be incorporated through an agreed process. Optional dashboards, advanced reporting, and other features not needed for the audit are excluded from the initial project.

The original budget is $85,000, covering software, hardware, and contracted work. Existing employee salaries are not charged to the project. The budget will be reviewed at each phase, with funds reserved for unexpected costs.

3. The Six Phases

The project follows the six phases described by Jewell (2026).

Phase	Activities, Responsibilities, and Deliverables
Initiation	The project manager works with the CIO to establish the goal, budget, timeline, and team responsibilities. The deliverable is a project charter approved by the CIO before proceeding.
Definition	The project manager gathers requirements from the IT director, Procurement, General Counsel, and the fourteen units. The deliverable is an approved requirements list identifying required software information, security needs, and audit expectations.
Design	The two IT developers prepare possible designs using an existing licensed platform. The project manager and IT director review the options. The deliverable is an approved design that meets the requirements and can be maintained by IT.
Development	The project manager and developers prepare system access, staff assignments, instructions, training materials, and the implementation schedule. The deliverable is a readiness checklist approved by the IT director confirming that implementation can begin without outstanding dependencies.
Implementation	The developers build the database while the student worker helps organize software records. The project manager coordinates the fourteen units. The deliverable is a functioning database tested against the approved requirements and piloted with two departments before university-wide use. The IT director approves acceptance testing.
Follow-up	The project manager transfers the system to the IT director, provides documentation, confirms maintenance responsibilities, and closes the budget. The deliverable is a signed handoff and final project report approved by the IT director and CIO.

Each phase will have a documented approval before the next major stage proceeds. This follows the handbook's approach of using phase boundaries to confirm progress and responsibilities (Jewell, 2026).

4. Stakeholder Table

Stakeholder	Interests and Possible Obstacles	Communication

CIO	Wants the inventory completed before the audit; controls project approval and major scope decisions.	Monthly 15-minute update
IT Director	Wants a reliable system that can be maintained after handoff; can reject an unmanageable solution.	Weekly meetings
IT Developers	Need clear requirements and enough time; competing assignments can delay implementation.	Twice-weekly check-ins
Procurement	Ensures purchasing and platform decisions follow university rules; can block unauthorized purchases.	At major decisions
General Counsel	Needs accurate licensing information for legal and audit purposes; can reject incomplete evidence.	Monthly and at approvals
Fourteen University Units	Must provide accurate software records; late or incomplete submissions can delay the project.	Biweekly updates
Spreadsheet Owner	Wants to keep control of the existing departmental inventory; may resist replacing the spreadsheet.	Weekly during integration
Student Worker	Helps organize and verify records; graduates at the end of month six.	Weekly check-ins

I would work directly with the spreadsheet owner instead of immediately requiring them to abandon their existing process. Their experience could help identify missing information and improve the new system.

5. Risk Register

The following risks are based on the university's actual staffing, procurement, and departmental constraints. The response categories follow the handbook's four approaches: avoiding, fighting, insuring, and accepting (Jewell, 2026).

Risk	Likelihood / Impact	Response and Action
Departments submit incomplete or late software records.	High / High	**Fight:** Provide templates, set deadlines, and follow up with unit representatives.

Spreadsheet owner refuses to cooperate.	High / High	**Fight:** Involve the employee early and allow the spreadsheet to remain in use while records are incorporated.
A new platform requires an eight-week procurement exception.	Medium / High	**Avoid:** Use only a platform the university already licenses.
Developers are reassigned to other university projects.	High / High	**Insure:** Reserve contingency funds for limited contracted assistance and prioritize essential work.
Student worker graduates before documentation is finished.	High / Medium	**Fight:** Complete data cleanup and knowledge transfer before the end of month six.
Additional features are requested after requirements are approved.	Medium / High	**Avoid:** Require formal approval for changes and defer optional features.
Audit requirements or deadlines change.	Medium / High	**Accept:** Escalate immediately to the CIO, revise the delivery plan, and prioritize required audit information.

The project manager will review these risks weekly and document changes in the risk register.

6. Original Nine-Month Timeline

The original schedule gives the team nine months to finish the database before the annual audit.

Phase	M1	M2	M3	M4	M5	M6	M7	M8	M9
Initiation	X								
Definition	X	X							
Design		X	X						
Development			X	X					
Implementation				X	X	X	X	X	
Follow-up									X

Critical Path: Requirements approval → platform and design approval → implementation readiness → database construction → pilot and testing → IT handoff.

These activities cannot be delayed without affecting the final completion date. The design depends on approved requirements, implementation depends on the completed preparations, and the final handoff depends on successful testing. Departmental data collection will begin during the earlier phases so the team does not have to wait until the database is finished to start gathering records.

7. Response to the Complication

At the end of month four, the CIO informed us that the audit was moved forward by two months and one developer would be reassigned to another project for six weeks. This reduces the original nine-month timeline to seven months and removes approximately three developer-weeks of work. Since the budget remains $85,000, I would focus on delivering an audit-ready version of the database rather than trying to complete the original scope with fewer resources.

Which Phase Absorbs the Change: The implementation phase would absorb most of the schedule reduction. Instead of spending months four through eight on implementation, the team would focus on finishing the essential database functions by month six, with final testing and handoff completed during month seven. The remaining developer would focus on the core database while the student worker continues organizing records. I would also evaluate using contracted assistance from the existing budget, provided the contractor can be approved and become productive quickly enough to help.

What Gets Cut or Delayed: I would remove the planned second round of university-wide usability improvements and limit the initial rollout to the functions required for the audit. Automated synchronization with the existing departmental spreadsheet would also be postponed. Instead, we would use a reviewed manual import process to make sure its records are included. Any additional automation or interface improvements would move to a separate second phase after the audit, managed by the IT director and subject to future approval.

What Gets Protected: The database must still contain the software inventory information required by Legal, including software names, versions, licensing information, departments, machines, and installation dates. I would also protect record verification, access controls, testing, and documentation. Reducing these activities could result in an unreliable database that does not actually solve the university's audit problem.

Risk Register Changes: The developer reassignment and earlier audit deadline activate two risks already identified in the register. I would update both to active issues, review the remaining workload, and determine whether the contingency budget can cover contracted assistance. I

would not assume additional money automatically solves the problem because contractors still require approval and onboarding.

Revised Timeline: Months one through four remain unchanged. During month five, the remaining developer would focus on the core database and data imports. During month six, the returning developer would assist with testing and corrections while the student worker completes their remaining documentation before graduation. Month seven would be reserved for final acceptance testing, audit reporting, and the handoff to the IT director.

Communication and Decisions: I would first meet with the IT director and developers to confirm the revised workload. Within the first week, I would give the CIO a short report explaining the reduced scope, revised schedule, and any request for contracted assistance. I would then inform General Counsel, Procurement, the fourteen departments, and the spreadsheet owner about the new deadlines and reporting requirements.

I would make it clear that I cannot guarantee the original full scope within seven months while also losing developer availability. I would say no to additional features, automated spreadsheet synchronization, and requests that do not directly support the audit. The university would receive a functional and verified software inventory by month seven, with additional improvements handled separately after the audit.

8. Where the Model Strains

The handbook's six phases provide a useful structure, but following them completely in order could create problems for this project. Waiting until implementation to begin collecting information from all fourteen departments would create unnecessary delays, especially if some departments respond slowly or provide incomplete records. I would keep the six phases for major approvals while allowing activities such as data collection, communication, and early testing to overlap. I would also use shorter testing cycles during implementation so the team can identify problems before the final month. This approach keeps the project organized while giving the team more flexibility when unexpected changes happen. Jewell (2026) explains that combining sequential planning with iterative work can help projects maintain accountability while adapting to changing conditions.

References

Jewell, T. (2026). *Project management: A handbook for information professionals* (Version 1.0) [Adapted from *Project management handbook* by W. Baars, 2006].